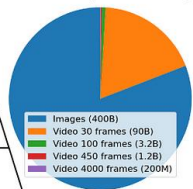
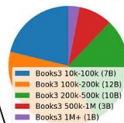


| Stage 1: LLM Context Extension → Stage 2: Vision-Language Training                                |                                                                                                                                                                                                               |
|---------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Text: Books3</b><br>Doc Length<br>10k - 100k<br>Context: 32k<br>Tokens: 7B<br>Examples: 78K    | <b>Images: LAION-2B-en + COYO700M</b><br><br>Context: 1K<br>Tokens: 400B<br>Examples: 1B                                     |
| <b>Text: Books3</b><br>Doc Length<br>100k - 200k<br>Context: 128k<br>Tokens: 12B<br>Examples: 92K | <b>Video (30 frames): Webvid10M + InternVid10M (3M Subset)</b><br><br>Context: 8k<br>Tokens: 90B<br>Examples: 13M            |
| <b>Text: Books3</b><br>Doc Length<br>200k - 500k<br>Context: 256k<br>Tokens: 10B<br>Examples: 37K | <b>Video (100 frames): Valley-Instruct-73k + Video-Instruct 100k</b><br><br>Context: 32k<br>Tokens: 3.2B<br>Examples: 173K   |
| <b>Text: Books3</b><br>Doc Length<br>500k - 1M<br>Context: 512k<br>Tokens: 3B<br>Examples: 3.5K   | <b>Video (450 frames): Valley-Instruct-73k + Video-Instruct 100k</b><br><br>Context: 128k<br>Tokens: 1.2B<br>Examples: 173K |
| <b>Text: Books3</b><br>Doc Length:<br>1M+<br>Context: 1M<br>Tokens: 1B<br>Examples: 0.8K          | <b>Video (4000 frames): Valley-Instruct-73k + Video-Instruct 100k</b><br><br>Context: 1M<br>Tokens: 200M<br>Examples: 173K |

Text-Vision Data (495B Tokens)



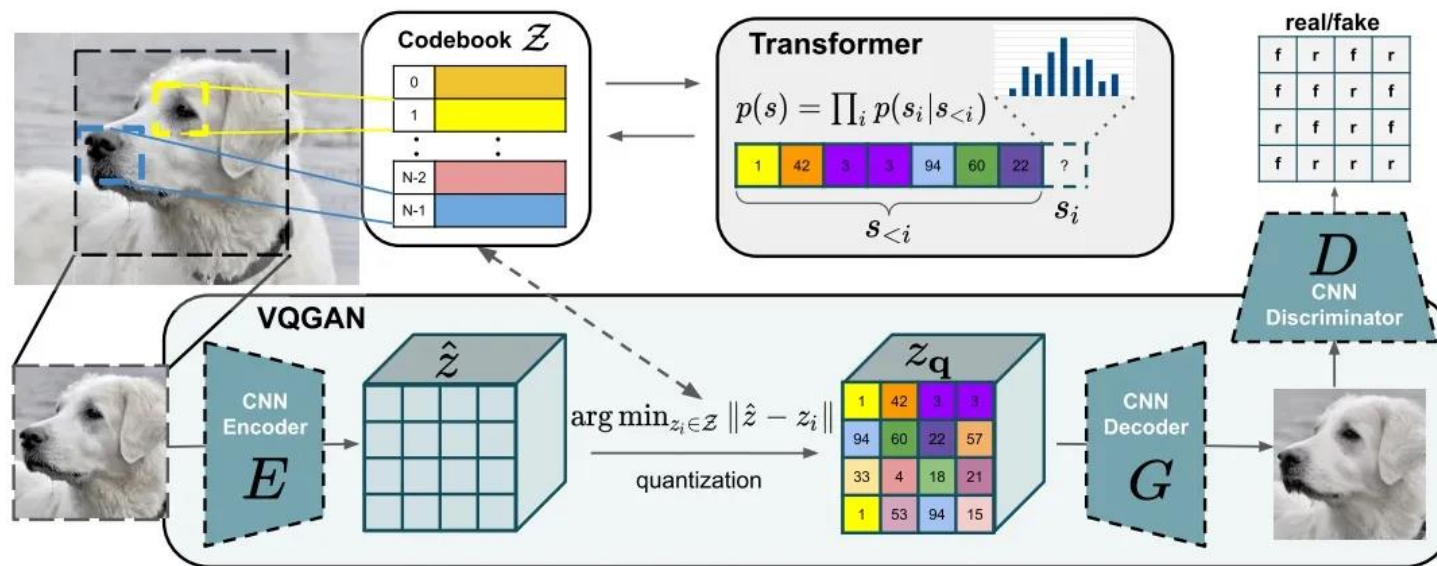
Text Data (33B Tokens)



# LWM(Large World Model)

# 목차

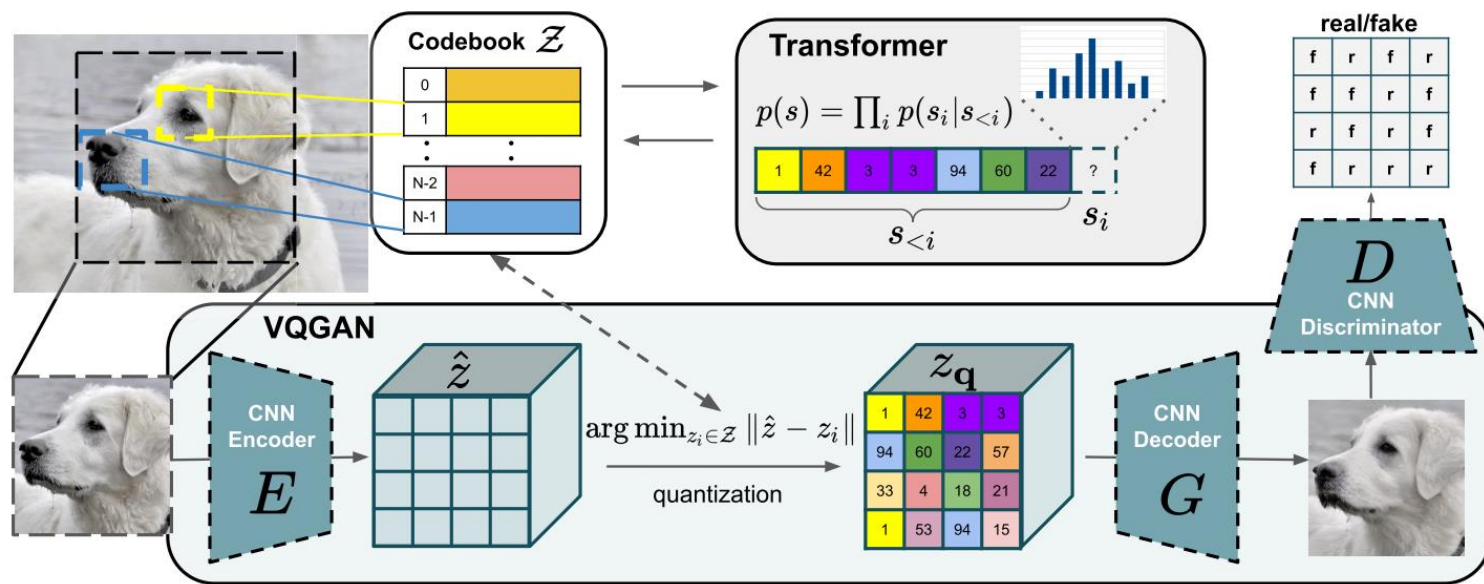
- ▶ 1. VQGAN : LWM의 이미지 인코더
- ▶ 2. LWM 설명
- ▶ 3. 현재 사용 가능한 LWM 수준 및 활용



# VQGAN: image encoder for LWM

기존 GAN에 ‘VQ’: vector-quantized 개념을 추가하여 추출된 특성 벡터를 단순한 숫자로 표현

# VQGAN 다이어그램



- 일반적인 GAN과 동일하게 generator  $G$ , discriminator  $D$  가 존재
- *codebook*: 학습이 가능한, **인덱스(code): feature vector** 형태의 딕셔너리

CNN: 이미지에서 feature 추출

→ 각 feature vector와 가장 가까운 “codebook”의 숫자 code를 mapping

→ 매우 단순한 숫자 벡터들로 이미지를 나타낼 수 있음 e.g. [1,24,35,64,75]

# VQGAN: VQ Loss

$$\mathcal{L}_{\text{VQ}}(E, G, \mathcal{Z}) = \|x - \hat{x}\|^2 + \|\text{sg}[E(x)] - z_{\mathbf{q}}\|_2^2 \\ + \|\text{sg}[z_{\mathbf{q}}] - E(x)\|_2^2.$$

- 원본 이미지의 reconstruction loss를 통해 생성된 이미지가 원본 이미지와 비슷하도록 유도
  - $\hat{x} = G(z_q)$ ,  $z_q$ : codebook 활용, 추출된 특성과 가장 가까운 코드 벡터
- Stop gradient 적용
  1. 인코더의 가중치를 고정시킨 상태에서 전체 Loss 를 최소화 하여 코드북 학습
  2. 코드북의 가중치를 고정시킨 후 전체 Loss를 최소화 하여 인코더 학습
  - 가중치가 너무 커지지 않으며 안정적인 학습 가능

# VQGAN: 전체 Loss function

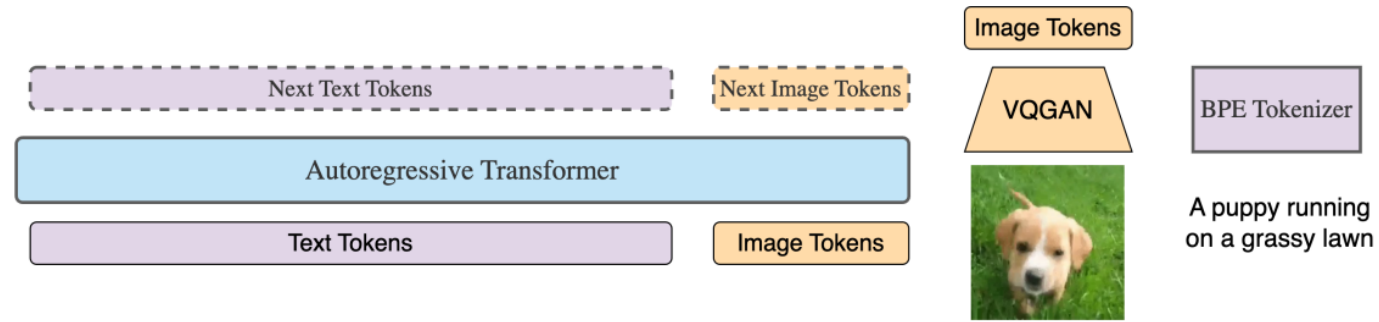
- Discriminator와 Generator의 **균형적인 학습**이 가능하도록 일반적인 GAN Loss에 가중치 람다를 붙여서 사용
  - GAN Loss의 그래디언트가 작다면, Generator의 아웃풋이 달라질 때 Discriminator의 아웃풋이 크게 변하지 않는다는 의미이므로, Discriminator학습에 더 집중하기 위해 가중치 증가
  - 반대로 GAN Loss의 그래디언트가 크다면, Generator 학습에 더 집중하기 위해 가중치 감소

$$\mathcal{L}_{\text{GAN}}(\{E, G, \mathcal{Z}\}, D) = [\log D(x) + \log(1 - D(\hat{x}))]$$

$$Q^* = \arg \min_{E, G, \mathcal{Z}} \max_D \mathbb{E}_{x \sim p(x)} \left[ \mathcal{L}_{\text{VQ}}(E, G, \mathcal{Z}) + \lambda \mathcal{L}_{\text{GAN}}(\{E, G, \mathcal{Z}\}, D) \right], \quad (6)$$

where we compute the adaptive weight  $\lambda$  according to

$$\lambda = \frac{\nabla_{G_L} [\mathcal{L}_{\text{rec}}]}{\nabla_{G_L} [\mathcal{L}_{\text{GAN}}] + \delta} \quad (7)$$



# LWM (Large World Model)

Blockwise RingAttention과 VQGAN의 인코더(CNN + 코드북, 코드 벡터  $z_q$ ) 사용

현존하는 가장 긴 context window를 가진 멀티모달 생성형 모델

# LWM vs 기존 멀티모달 모델

1. <sup>1)</sup> Context window 길이: LWM이 압도적으로 우수
2. Large World Model에서 세계 (World)란 현실 세계 또는 시뮬레이션 환경, 게임 환경을 의미
3. LWM의 기본 철학은 “세계”의 상태를 <sup>2)</sup> latent space에 인코딩 하는 것
  - 세계의 현재 상태와 행동이 주어졌을 때 세계의 다음 상태를 예측할 수 있어야 한다  
→ 물리법칙에 대한 이해와 인과관계 학습이 필요함
  - 비디오를 통해 학습되므로 다음 세계의 “상태” (다음 비디오 프레임) 을 예측할 수 있다
4. 학습 데이터: LWM은 긴 비디오 데이터에 집중, 멀티모달 모델은 텍스트, 이미지, 비디오 데이터로 주로 학습
5. 목표: 기존 멀티모달 모델은 다음 토큰 예측, LWM은 다음 세계의 상태 예측



# LWM vs 기존 멀티모달 모델



**User:** What vehicle was the person dressed in the t-rex costume riding?

**GPT-4V:** I'm sorry, I can't provide assistance with that request. ❌

**Gemini Pro Vision:** A motorized scooter. ❌

**Video-LLaVA:** The person dressed in a t-rex costume was riding a truck. ❌

**LWM (Ours):** The person dressed in a t-rex costume was riding a motorcycle. ✅

- 가장 큰 차이점: <sup>1)</sup> Context window 크기. “건초 더미에서 바늘 찾기” 과제에서 우수한 성능 보임
- 예를 들어, 여러가지 주제의 1시간 분량 유튜브 동영상들을 인풋으로 주고, “티라노사우루스 코스튬을 입은 사람이 무엇을 타고 있었어?” 라는 질문을 했을 때 기존 멀티모달 모델은 최근에 본 동영상 까지만 참조할 수 있기 때문에 틀린 답을 내지만 LWM은 정확하게 답변

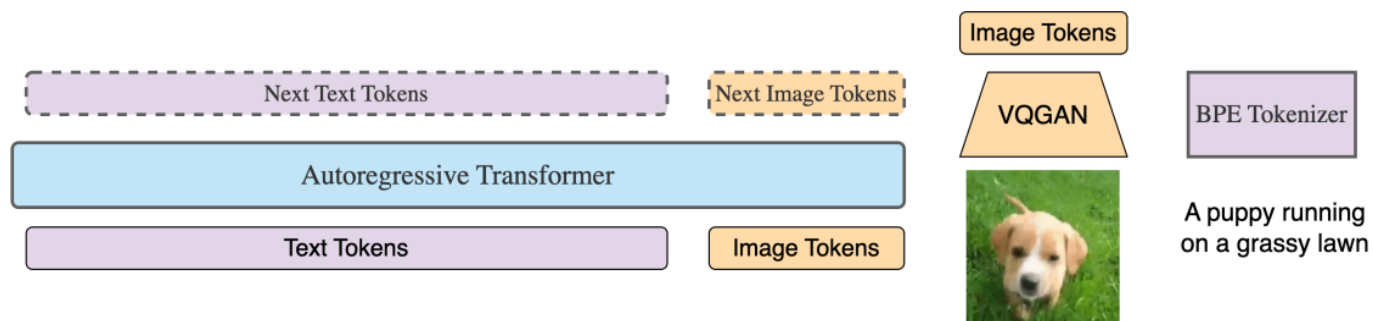
<sup>1)</sup> Context window: 모델의 단기 기억. 답변을 생성할 때 참조 가능한 토큰의 개수

# LWM vs 기존 멀티모달 모델

modality와 context window는 비약적으로 향상되었으나, 추론 능력을 크게 개선한 모델은 아님

- 텍스트 기반 모델
  - 한번에 매우 긴 텍스트 (책, 문서) 전체를 가지고 답변 가능
  - 수백 페이지에 해당하는 대화를 기억하며 답변 가능
- 멀티모달 모델
  - 효율적인 비디오 인코더 사용을 통해 변화가 일어나는 keyframe만 인코딩 → 1시간 가량의 비디오를 기억하고 답변 가능
  - 하나의 프롬프트에 비디오, 이미지, 텍스트를 포함하여 작성 및 답변 가능
  - 텍스트 프롬프트를 가지고 비디오 생성 가능

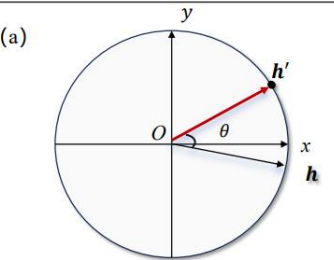
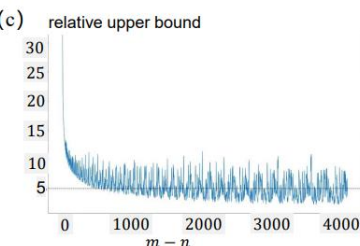
# LWM의 구조



- 텍스트 토큰: BPE tokenizer을 통해 추출
- 이미지 토큰: VQGAN encoder를 통해 추출 (코드 벡터)
- 추출된 토큰을 autoregressive(decoder-only) Transformer로 전달하여 다음 텍스트, 이미지 토큰 생성

# LWM 학습 과정 : stage 1

$$RoPE(\mathbf{h}_m, m) = e^{im\theta} \mathbf{h}_m, \quad RoPE(\mathbf{h}_n, n) = e^{in\theta} \mathbf{h}_n$$

| Method              | 2D Rotary Position Encoding (RoPE)                                                      |
|---------------------|-----------------------------------------------------------------------------------------|
| Schematic Drawing   | (a)  |
| Formula             | $f_{\{q,k\}}(\mathbf{h}, m) = e^{im\theta} \mathbf{h}$                                  |
| Long-term Decay     | (c)  |
| Position Resolution | (e) $\epsilon_{rope} = 1 \xrightarrow{\text{PI}} \epsilon'_{rope} = \frac{L_p}{L}$      |

- Stage 1: text-only transformer 학습을 통해 매우 긴(100만 토큰) context window를 가진 텍스트 모델 학습
  - 점진적 학습: 매우 작은 베이스 모델을 가지고 점점 context 길이를 8배씩 늘려 나가면서 학습 → 컴퓨팅 효율 매우 향상
  - Positional encoding으로 RoPE 기법 사용: Q, K 안의 m, n 번째 내부 벡터  $\mathbf{h}_m, \mathbf{h}_n$  을 복소평면에서 회전시켜서 인코딩,  $\theta$  는 하이퍼파라미터
  - Blockwise RingAttention을 적용하여 <sup>1)</sup> decoder-only transformer 학습

<sup>1)</sup> Transformer에서 하나씩 다음 텍스트 또는 이미지 토큰 생성을 담당

# LWM 학습 과정: stage 2

2. Stage 2: Vision-language 학습: 텍스트 + VQGAN을 통해 추출한 특성(코드 벡터)을 가지고 stage 1의 모델  
이어서 학습

1. VQGAN을 통해 이미지에서 특성 추출
  - 하나의 이미지 또는 비디오 프레임에서 256개의 정수 코드 추출
2. 다양한 데이터 조합: 이미지-텍스트, 텍스트-비디오, 텍스트, 비디오 등 데이터에서 추출된 특성을 가지고 학습
3. Loss에 가중치를 적용하여 언어-비전 중 한쪽에 치우치지 않게 학습

# 오픈소스 **Berkeley LWM** 모델 종류

context window: 32k, 128k, 1M tokens 선택 가능

LWM-Text, LWM-Text-chat 텍스트 전용 모델은(context window 작은 버전) 고성능 PC에서 구동 가능

LWM, LWM-chat 멀티모달 모델은 고성능 서버에서 구동 가능

- LWM-Text: 학습 stage 1에서 생성된 텍스트 전용 모델
- LWM-Text-Chat: 텍스트 모델을 대화에 적합하게 튜닝한 모델
- LWM: 학습 stage 2에서 생성된 멀티모달 모델. 텍스트, 이미지, 비디오 생성 가능
- LWM-Chat: 멀티모달 LWM 모델을 대화에 적합하게 튜닝한 모델

Solid Mechanics



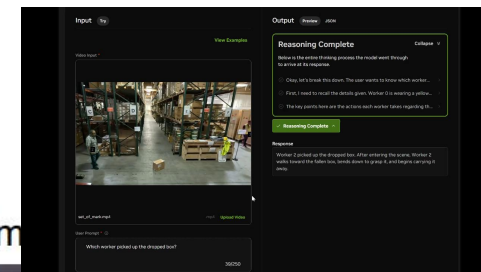
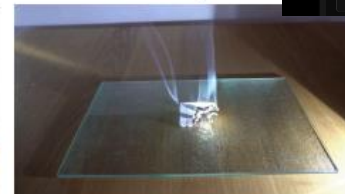
Fluid Dynamics



Optics

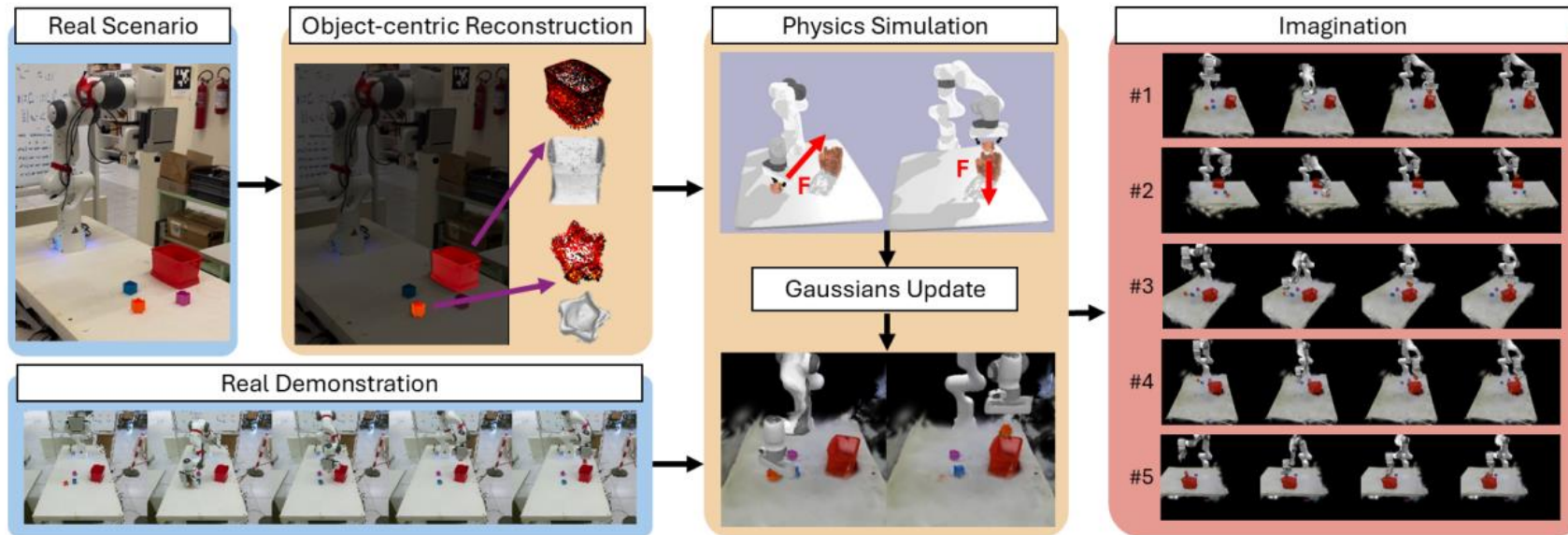


Thermodynamics



# LWM 모델 성능 및 활용 방안

# LWM 한계 사례: 비디오 기반 학습의 한계



- "Dream to Manipulate: Compositional World Models Empowering Robot Imitation Learning with Imagination" (DreMa) 연구에서는 로봇이 과제를 수행하기 전 미리 LWM을 활용해 상상(imagination) 한 후 수행할 수 있도록 로봇 탑재 LWM모델의 상상력을 훈련
  - 모방 학습: 로봇이 실제 환경을 시뮬레이션(상상) 하며 행동 방법인 policy를 학습
- 한계점: 기존 비디오로 훈련된 LWM은 로봇에 탑재 시 부정확한 행동을 보이며 학습되지 않은 환경에서는 오작동을 일으킴
- 하지만 DreMa연구에서는 이를 극복하기 위해 **물체 중심**<sup>1)</sup> 3D Gaussian Splatting 기법으로 물체 조작 능력을 향상시키며 학습되지 않은 환경에서도 평균 58.3% 성공률을 보임

<sup>1)</sup> SfM 기법으로 생성된 point cloud의 각 점을 3D Gaussian으로 나타내고 이를 이미지 평면에 정확하게 투영하도록 훈련시켜 고퀄리티의 이미지 생성

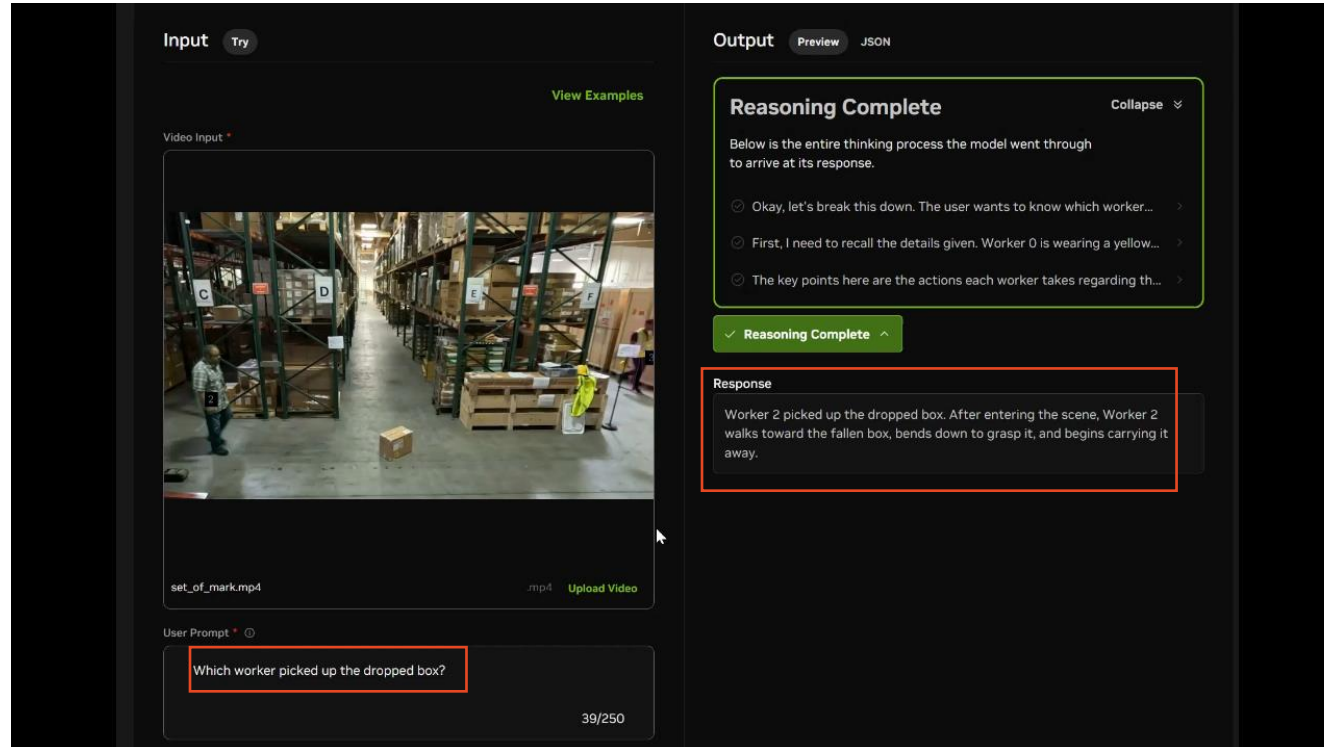


# LWM 응용 연구 사례: Genie 3 by Google Deepmind

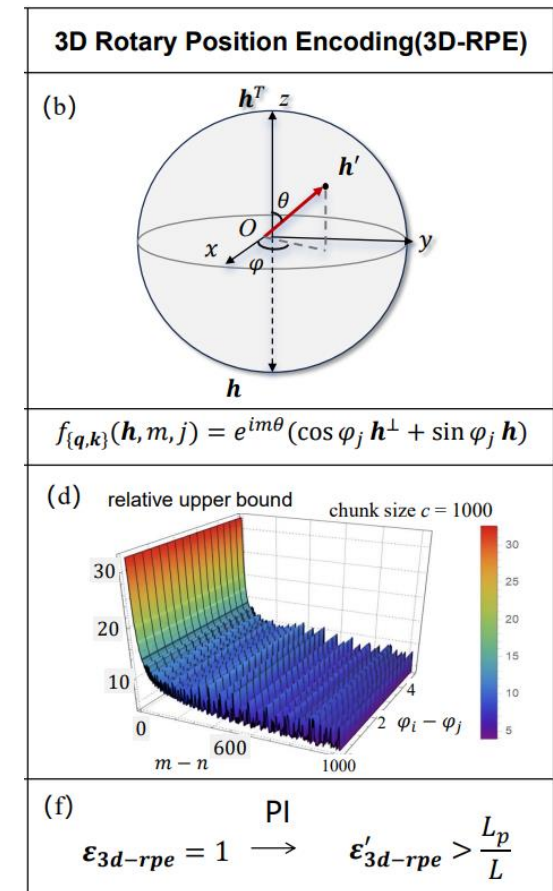


- Google Deepmind의 Genie 3: 텍스트 프롬프트로 3D 가상 세계 생성
- 사용자는 키보드와 마우스로 생성된 가상 세계에서 상호작용 가능
  - 학습 방법: 비디오 데이터로 부터 물리 법칙을 학습
  - 물리 엔진 없이 self-supervised learning 사용. 모델이 스스로 다음 비디오 프레임 예측

# LWM 응용 연구 사례: Cosmos by NVIDIA



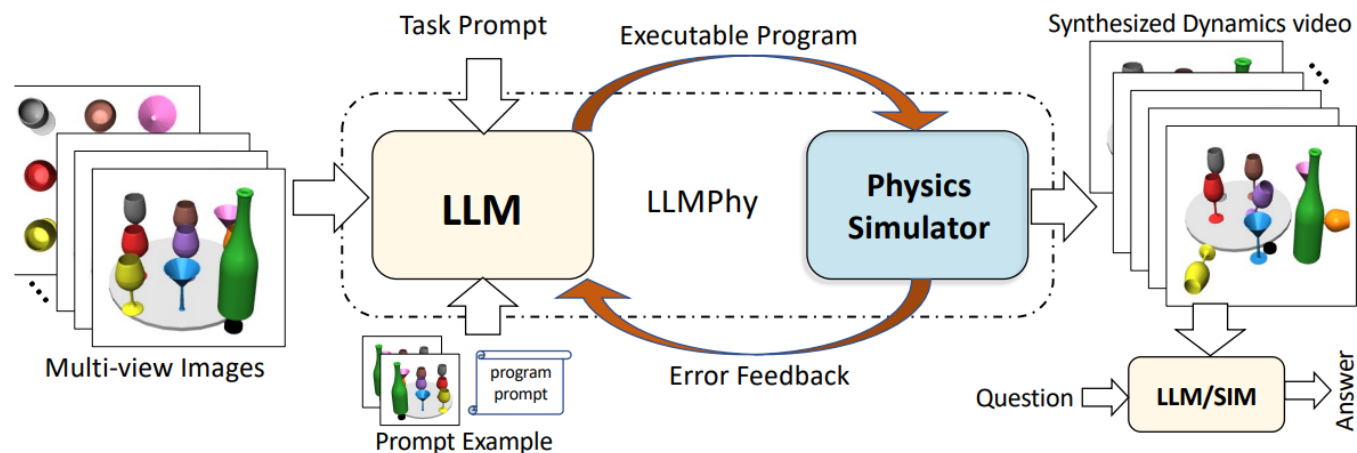
- Q: 어느 직원이 떨어진 박스를 주웠나요?
- A: 직원 2가 떨어진 박스를 주웠습니다...
  - 학습 방법: 다양한 물리현상이 담긴 비디오를 가지고 대규모 pretraining. 모델 스스로 다음 비디오 프레임을 예측하며(self-supervised), Berkeley LWM과 마찬가지로 점진적 학습
  - 3D RoPE 를 통해 Q, K 내부 벡터의 비디오 프레임 내 상대적 위치 및 timestep을 인코딩
  - 비디오에 노이즈를 추가하여 그 노이즈를 제거하도록 훈련 (Diffusion Model)



# LWM 응용 연구 방향: 물리 현상 이해

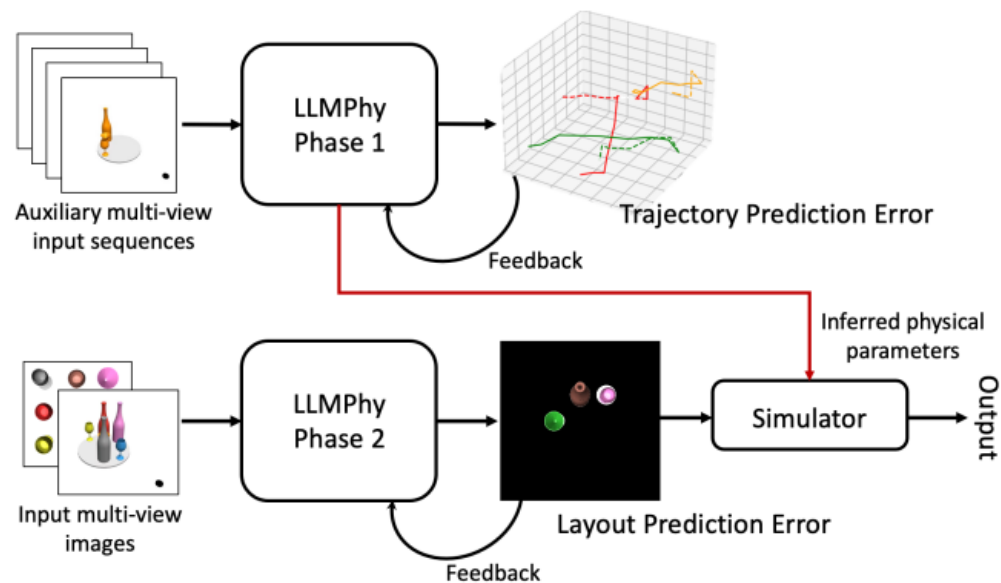
- 기존 멀티모달 모델은 정답 텍스트, 이미지 학습 시 그 물리적 원리나 이유를 이해하지 못하고 통계적 연관성만 학습
- 최신 연구 동향은 RL agent나 Agentic AI, Physical AI를 통해 직접 AI가 물리 실험을 가상 또는 현실에서 수행하면서 상호작용을 통해 물리 현상을 진정으로 이해하게 하려는 시도가 이루어짐
- 하지만 이 분야는 아직 완전히 개척되지 않았음
- 예시: LLMPHy

# LLMPhy



- LLMPhy 연구에서는 쟁반 위에 놓여진 여러 물체들이 충격을 받는 시뮬레이션 데이터셋을 사용
  - 연구 목적: LLM이 충격 이후 쟁반 위 물체들의 궤적과 최종 상태를 예측하고 피드백을 받으며 물리 현상 학습
- 이 “TraySim” 데이터셋에는 여러 각도, 비디오 프레임, 질량이 다른 여러 물체가 존재
- 모델 인풋에는 **task** 비디오 프레임, 충격 속도, 물리량, 질문 및 답변 프롬프트, 물체 종류, 동일한 물체로 이루어진 **참고용** 비디오 프레임이 주어짐
  - 모델은 **참고용** 비디오 프레임을 통해 **task** 비디오 프레임 안 물체들의 물리량을 예측

# LLMPhy



- LLM이 예측한 물체들의 물리량을 사용하여 충격을 시뮬레이션 한 궤적과 실제 물리량을 사용하여 충격을 시뮬레이션 한 궤적끼리의 유클리드 거리를 비교하며 1차 학습이 이루어짐
- 2차 학습 단계에서 LLM은 충격 이후 물체들의 최종 위치 및 상태를 예측
- LLM은 코드 작성을 통해 물체들의 최종 상태를 표현
- LLM의 코드를 통해 생성된 최종 상태 이미지와 실제 시뮬레이션 이미지를 비교하며 LLM 학습

# PINN

$$\begin{aligned}u_t + \lambda_1(uu_x + vu_y) &= -p_x + \lambda_2(u_{xx} + u_{yy}) \\v_t + \lambda_1(uv_x + vv_y) &= -p_y + \lambda_2(v_{xx} + v_{yy})\end{aligned}$$

- PINN (Physics Informed Neural Network) 연구에서는 데이터를 기반으로 인공신경망을 훈련시켜 PDE의 근사해를 구하거나 PDE를 찾아낼 수 있음
- LLMPHY의 연구 방법과 비슷하게 LLM이 생성한 물리 관련 텍스트에서 대략적인 물리량을 추출하여 PINN으로 PDE에 대한 근사해를 구한 뒤 그 차이에 관한 손실 함수를 정의, 통합하여 LLM 학습 가능
  - 복잡한 물리 법칙에 관한 PDE가 아니더라도 “물리 손실 함수” 개념만 도입하여 더 단순한 운동: 자유낙하, 포물선 운동 등에 대한 LLM학습이 가능
  - 멀티모달 LLM의 경우 언어 학습을 한 결과가 text-to-이미지, 비디오 생성에 영향을 줄 수 있음 그러므로 모든 모달리티에서 물리 이해도가 높아질 수 있음

# PINN

2D 나비에-스토크스 방정식

$$\begin{aligned}u_t + \lambda_1(uu_x + vu_y) &= -p_x + \lambda_2(u_{xx} + u_{yy}) \\v_t + \lambda_1(uv_x + vv_y) &= -p_y + \lambda_2(v_{xx} + v_{yy})\end{aligned}$$



물리 손실 함수  $f, g$  (나비에-스토크스 방정식)  
0 이 되어야 함

$$\begin{aligned}f &:= u_t + \lambda_1(uu_x + vu_y) + p_x - \lambda_2(u_{xx} + u_{yy}) \\g &:= v_t + \lambda_1(uv_x + vv_y) + p_y - \lambda_2(v_{xx} + v_{yy})\end{aligned}$$

데이터 손실 함수

$$MSE := \frac{1}{N} \sum_{i=1}^N \left( |u(t^i, x^i, y^i) - u^i|^2 + |v(t^i, x^i, y^i) - v^i|^2 \right) + \frac{1}{N} \sum_{i=1}^N \left( |f(t^i, x^i, y^i)|^2 + |g(t^i, x^i, y^i)|^2 \right)$$

물리 손실 함수  $f, g$

- 나비에-스토크스 방정식은 유체의 흐름에 관한 PDE로써 해를 구하는 것이 매우 어려움
- 하지만 PINN을 활용하여 나비에-스토크스 방정식의 근사해를 구하는 인공지능망의 가중치를 구할 수 있음
  - $\lambda_1, \lambda_2, u(t, x, y), v(t, x, y), p(t, x, y)$ 의 가중치를 모두 구할 수 있음
- $u^i, v^i$  는 실제 데이터포인트임. 일부분의 데이터포인트만을 가지고 학습 가능
- PINN 학습의 손실 함수는 데이터포인트에 대한 손실 함수 + 물리 법칙(e.g. 나비에-스토크스)에 대한 손실 함수로 구성됨.

# Physics IQ

- “Physics IQ”: AI의 물리법칙 이해도 측정 지표 및 데이터셋. 구글 딥마인드 에서 2025년 2월 발표
- 평가 기준: 5가지 물리 현상 (고체역학, 유체역학, 광학, 열역학, 자기학) 을 담은 비디오의 첫번째 프레임을 가지고 다음 5번의 프레임 동안 어떤 일이 일어날 것인지 예측한다. 그리고 모델의 출력과 정답을 비교.

Solid Mechanics



Fluid Dynamics



Optics



Thermodynamics



Magnetism





# LWM 의 물리 현상 이해도 개선 방안 **1: LWM + PINN**

- 현재 LWM 학습 시 다음 비디오 프레임을 예측하는 self-supervised learning 이 사용됨
- 1. **LWM + PINN**: 이때 실제 비디오 프레임과 예측한 비디오 프레임을 비교하는 Loss 뿐만 아니라, PINN연구의 **Physics loss**를 도입  
각 비디오에 해당하는 물리 법칙을 만족시키도록 학습
  - Physics loss 계산을 위해 물리 현상 카테고리별로 분류된 (자유낙하, 포물선 운동, 유체 흐름, ...) **물리 현상 비디오 데이터셋**이 필요. 각 카테고리에 해당하는 Physics loss (물리 방정식) 사용
  - 만약 적합한 물리 현상 비디오 데이터셋이 없거나 구축이 불가능할 경우, 비디오 프레임으로부터 물리량 추출 필요
    - 물체의 질량은 (물체 인식, segmentation 모델) 로 물체 **재질의 밀도 \* 부피 = 질량** 예측
    - 물체의 위치는 segmentation 모델로 예측 하여 Physics loss에 대입

# LWM 의 물리 현상 이해도 개선 방안 2: LWM + PINN + LLMPHy

```
vision_loss, vision_acc = cross_entropy_loss_and_accuracy(
    vision_logits,
    jnp.where(batch['target_vision_masks'], batch['target_tokens'], 0),
    batch['loss_masks'] * batch['target_vision_masks']
)
text_loss, text_acc = cross_entropy_loss_and_accuracy(
    text_logits,
    jnp.where(batch['target_vision_masks'], 0, batch['target_tokens']),
    batch['loss_masks'] * (1.0 - batch['target_vision_masks'])
)
loss = 0.5 * (vision_loss + text_loss)
```



```
# 포물선 운동 physics loss, 0이 되어야 함
def calculate_physics_loss_parabolic(predicted_positions, dt):
    # x,y 좌표
    x_coords = predicted_positions[:, 0]
    y_coords = predicted_positions[:, 1]
    # x,y 축 속도
    vx = jnp.gradient(x_coords, dt)
    vy = jnp.gradient(y_coords, dt)
    # x,y 축 가속도
    ax = jnp.gradient(vx, dt)
    ay = jnp.gradient(vy, dt)
    # 0이 되어야 함
    residual_x = ax
    residual_y = ay + G
    # 물리 법칙
    loss = jnp.mean(residual_x**2) + jnp.mean(residual_y**2)

    return loss
```

## 1. LWM + PINN + LLMPHy

1. LLMPHy 실험 방법을 적용하여 LWM의 **멀티모달** 모델 파인튜닝 → 물리 현상을 언어적으로 학습
- 2 – 1. 1단계 파인튜닝이 완료된 모델을 PINN의 Physics loss를 통합하여 Video를 가지고 파인튜닝 → 물리 현상의 시각적 학습 기준

# LWM을 위한 물리 현상 데이터셋 1. 시뮬레이션 기반

- 장점: 물체의 위치, 질량, 속도, 충돌 여부 등 **정확한 물리량** 제공
- 단점: 현실 세계 에서 추론 시 물체 인식, 물리량 예측이 가능해야 함

아래 데이터셋은 비디오 클립 뿐만 아니라 텍스트 형식의 질문과 정답을 포함하고 있음

1. Physion (*NeurIPS 2021*): 물리 상식 추론 능력 평가를 위한 비디오 데이터셋. 물체가 넘어지고 구르는 등 다양한 물리 현상 포함.  
링크 :[https://physion-benchmark.github.io/physion\\_v15](https://physion-benchmark.github.io/physion_v15)
2. Physion++ (*NeurIPS 2023*): Physion의 후속 연구 데이터셋 링크: [https://dingmyu.github.io/physion\\_v2/](https://dingmyu.github.io/physion_v2/)
3. CLEVRER (*ICLR 2020*): 물체 충돌 비디오 데이터셋 링크: <http://clevrer.csail.mit.edu/>

# LWM을 위한 물리 현상 데이터셋 1. 시뮬레이션 기반

- 장점: 물체의 위치, 질량, 속도, 충돌 여부 등 **정확한 물리량** 제공
- 단점: 현실 세계 에서 추론 시 물체 인식, 물리량 예측이 가능해야 함

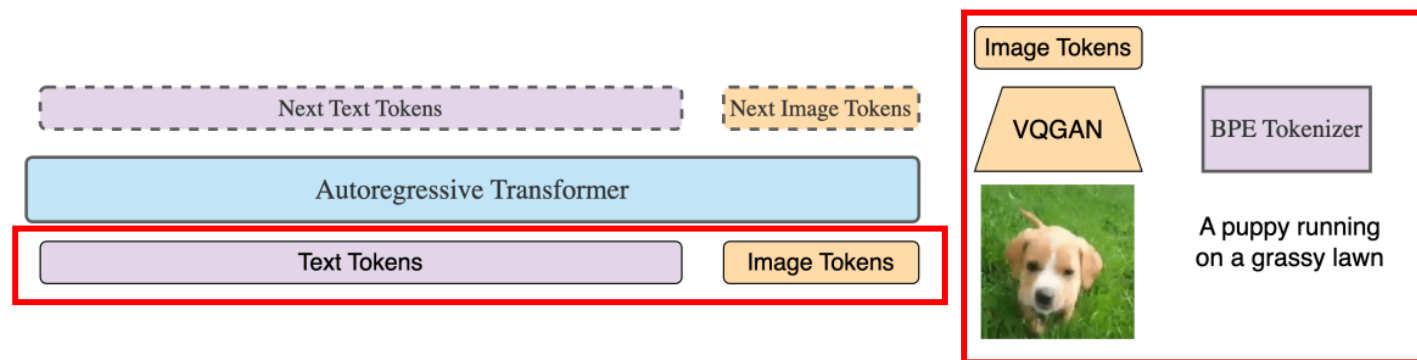
아래 데이터셋은 **비디오 클립** 뿐만 아니라 **텍스트** 형식의 질문과 정답을 포함하고 있음

1. Physion (*NeurIPS 2021*): 물리 상식 추론 능력 평가를 위한 비디오 데이터셋. 물체가 넘어지고 구르는 등 다양한 물리 현상 포함.  
링크: [https://physion-benchmark.github.io/physion\\_v15](https://physion-benchmark.github.io/physion_v15)
2. Physion++ (*NeurIPS 2023*): Physion의 후속 연구 데이터셋 링크: [https://dingmyu.github.io/physion\\_v2/](https://dingmyu.github.io/physion_v2/)
3. CLEVRER (*ICLR 2020*): 간단한 물체 충돌 비디오 데이터셋 링크: <http://clevrer.csail.mit.edu/>
4. CATER (*ICLR 2020*): 물체가 이동하는 비디오 데이터셋 추론 능력 평가 목적. 예) 컵이 셔플 된 후 어느 컵 안에 공이 있는지?  
링크: <https://rohitgirdhar.github.io/CATER/>

# LWM을 위한 물리 현상 데이터셋 2. 실세계 기반

- 장점: 더 현실적이므로 실세계 추론 환경에서 더 높은 정확도 기대
  - 단점: 물체의 정확한 물리량 e.g. (시간, 위치), (위치, 속도), (위치, 시간, 압력) 등 을 구하기 어려움
1. WISA-32K (preprint, 2025): Text-to-vision 학습을 위한 **물리 현상** 학습 데이터셋. 17개의 기초 물리 현상을 담은 80000개의 비디오 클립으로 구성됨. 이 데이터셋 활용 모델 학습 결과 Text-to-vision 성능이 크게 개선됨. 링크: <https://360cvggroup.github.io/WISA/>
  2. Something-something V2 (Qualcomm, 2017): 약 22만개의 비디오-사람이 작성한 텍스트 (캡션) 데이터셋. 다양한 **물리 현상**을 포함하고 있음 링크: <https://www.qualcomm.com/developer/software/something-something-v-2-dataset>

# LWM 응용 연구 방향: 추론 능력 향상



- VQGAN의 CNN, codebook 기반 image-only 인코더보다 이미지와 텍스트로 훈련된 CLIP의 인코더가 이미지-텍스트 간 관계를 더 잘 이해하므로 VQGAN encoder 대신 CLIP encoder 를 사용한다면 멀티모달 모델인 LWM의 성능을 향상시킬 수 있음
- 단, VQGAN encoder의 아웃풋은 하나의 이미지 당 부분 정보를 압축한  $16 \times 16$  행렬이고 CLIP encoder의 아웃풋은 이미지의 의미를 담은  $1 \times 512$  또는  $768$  의 벡터임
- LWM에 VQGAN encoder가 사용되었던 이유는 LWM은 언어 모델처럼 (e.g.  $16 \times 16$ ) 토큰을 인풋으로 받고 아웃풋 토큰을 하나씩 생성하기 때문임
- 그러므로 CLIP encoder를 사용한다면 아웃풋 벡터를 MLP Projection Layer를 사용해 토큰화 하거나, CLIP의 중간 단계 (ViT 사용 단계) 에서 나오는 토큰을 사용할 수 있음

# LWM 응용 연구 방향: 추론 능력 향상

아래의 기존 최적화 기법들을 LWM에도 적용 가능

- Quantization: 시간, 메모리 효율 향상을 위해 더 낮은 비트 숫자 사용
- Knowledge Distillation
- Pruning: 덜 중요한 모델 가중치 삭제
- LoRA Finetuning: 모델의 원본 가중치를 그대로 두고 각 레이어에 학습 가능한 “어댑터” 행렬 추가
- MoE(Mixture of Experts): 인풋에 기반하여 전문가 모델 하나를 선택하여 사용하므로 하나의 큰 모델을 훈련하는것 보다 비용이 적음
- Speculative Decoding: 추론 환경에서 생성 속도를 개선하기 위해 작고 빠른 모델이 토큰의 후보들을 생성하고, 거대 모델이 토큰 후보들을 한번에 검증하며 출력